



Universidad
Nacional
de Córdoba



Facultad de Matemática,
Astronomía, Física y
Computación

Proyecto: “Análisis Discriminante Lineal (LDA)”

ANÁLISIS NUMÉRICO II - 2024

Rubén Alfredo Mendoza

13 de Noviembre de 2024

Resumen

En este proyecto se utilizó el Análisis Discriminante Lineal (LDA) para clasificar tumores en dos categorías, benignos o malignos, utilizando el conjunto de datos "Breast Cancer" de la librería `scikit-learn`. Tras dividir los datos en entrenamiento y prueba, y estandarizar las características, se calcularon las matrices de dispersión intra e interclase para determinar la dirección que maximiza la separación entre las dos clases. Proyectando los datos en esta dirección, se programó un clasificador simple basado en la distancia a las medias proyectadas de cada clase. El método mostró un buen rendimiento, alcanzando una exactitud del 95.61 % sobre los datos de prueba, lo que evidencia su eficacia en la reducción de dimensionalidad y la clasificación de datos.

1. Introducción

En este trabajo se estudió el uso del Análisis Discriminante Lineal para reducir la dimensionalidad de un conjunto de datos con múltiples características, y la clasificación de los mismos según una variable cualitativa. En este caso, el objetivo fue clasificar datos de tumores en dos clases: benignos o malignos.

El Análisis Discriminante Lineal es una poderosa técnica para maximizar la separabilidad entre clases al proyectar los datos en una dirección óptima. Se basa en resolver un problema de autovalores para encontrar las direcciones de proyección que maximizan la razón entre la dispersión inter-clase y la dispersión intra-clase. Al final, se proyectan los datos en un nuevo subespacio generado por los autovectores, donde la maximización de la separación entre clases, facilita la clasificación de los datos.

Los datos fueron obtenidos del conjunto de datasets de prueba que se encuentran disponibles en la librería `scikit-learn`. Esta base de datos incluye características cuantitativas de tumores, como la textura, el área o la simetría del núcleo celular, que sirven para diferenciar entre tumores benignos y malignos.

Este procedimiento puede aplicarse en diversas áreas, como el reconocimiento de voz, clasificación de imágenes y muchos otros problemas de clasificación supervisada. La gran ventaja del

Análisis Discriminante Lineal es que puede tener un rendimiento igual o superior a otros modelos de aprendizaje supervisado usados para clasificación, aún con pocos datos de entrenamiento.

2. Trabajo realizado

Este trabajo se dividió en 4 etapas:

- 1) Preprocesamiento de los datos.
- 2) Cálculo de Medias y matrices de Dispersión.
- 3) Búsqueda de las direcciones que maximicen la separabilidad entre clases.
- 4) Proyección y clasificación.

A continuación procedemos a explicar mas detalladamente en que consisten cada uno de estos pasos. Todos los códigos utilizados se encuentran anexados en la sección [Apéndice](#), al final de éste documento.

2.1. Preprocesamiento de los Datos

En esta primera etapa, se realizaron las siguientes tareas:

- **División del Conjunto de Datos:** Los datos fueron divididos en un conjunto de entrenamiento y un conjunto de prueba utilizando el método `train_test_split` de `sklearn`. Esto se hizo para obtener un conjunto de datos que sirvan para evaluar de manera objetiva el rendimiento del clasificador.
- **Escalado de Datos:** Se aplicó una normalización de las características utilizando el `StandardScaler` de `sklearn` para asegurar que las variables tuvieran una varianza similar. Esto permitió asegurar que todas las variables tuvieran la misma escala y evitar que las diferencias de magnitudes entre las características afectaran el desempeño del clasificador.

El proceso de estandarización de los datos se realizó aplicando la siguiente fórmula para cada característica x_i :

$$X_i = \frac{x_i - \mu_i}{\sigma_i} \quad (1)$$

Donde:

- x_i es el valor original de la característica.
- μ_i es la media de la característica i en el conjunto de entrenamiento.
- σ_i es la desviación estándar de la característica i en el conjunto de entrenamiento.
- X_i es el valor estandarizado de la característica i .

2.2. Cálculo de Medias y Matrices de Dispersión

En esta etapa, se calcularon las matrices de dispersión dentro de cada clase y entre las clases para obtener una representación de la variabilidad de los datos. Esto permite caracterizar cómo se distribuyen las clases en el espacio de características.

- **Cálculo de medias:** El primer paso consiste en calcular la media de cada clase y la media global. La media de una clase es el vector de características promedio para los datos de esa clase, mientras que la media global es el promedio de todas las muestras independientemente de su clase.

$$\mu_i = \frac{1}{N_i} \sum_{j=1}^{N_i} X_{ij} \quad (2)$$

$$\mu = \frac{1}{N} \sum_{i=1}^K \sum_{j=1}^{N_i} X_{ij} \quad (3)$$

Donde:

- μ_i es la media de la clase i ,
 - N_i es el número de muestras en la clase i ,
 - X_{ij} es el vector de datos j en la clase i ,
 - μ es la media global de todos los datos,
 - K es el número total de clases,
 - N es el número total de muestras en todas las clases.
- **Matriz de dispersión intra-clase:** Esta matriz mide la variabilidad de los datos dentro de cada clase. Se calcula como la suma de las covarianzas de cada clase, utilizando los datos de cada clase en relación con su media. Cuanto menor sean los valores de esta matriz, más homogénea será la clase y menos dispersos estarán los datos dentro de ella.

$$S_{\text{intra}} = \sum_{i=1}^K \sum_{j=1}^{N_i} (X_{ij} - \mu_i)(X_{ij} - \mu_i)^T \quad (4)$$

- **Matriz de dispersión entre-clase:** Esta matriz mide la variabilidad de los datos entre las clases. Se calcula considerando la media global de todos los datos y las medias de cada clase, ponderando la distancia entre ellas por el número de muestras en cada clase. Cuanto mayor sean los valores esta matriz, mayor será la separación entre las clases.

$$S_{\text{entre}} = \sum_{i=1}^K N_i (\mu_i - \mu)(\mu_i - \mu)^T \quad (5)$$

2.3. Búsqueda de las Direcciones que Maximicen la Separabilidad entre Clases

El objetivo de esta etapa fue identificar las direcciones en el espacio de características que mejor separen las clases. Esto se logra mediante el cálculo de los autovectores de la matriz

$S_{\text{intra}}^{-1}S_{\text{entre}}$, que representa la relación entre la dispersión dentro de las clases y la dispersión entre ellas.

A continuación se describen los pasos realizados para encontrar estas direcciones de máxima separabilidad:

■ **Cálculo de la Inversa de la Matriz de Dispersión Intra-clase:**

Antes de calcular los autovectores y autovalores, es necesario obtener la inversa de la matriz de dispersión intra-clase (S_{intra}^{-1}). Para calcular la inversa de S_{intra} , se empleó el método de descomposición LU con permutaciones. A continuación, se resolvieron sistemas lineales mediante sustitución hacia adelante y hacia atrás.

■ **Cálculo de los Autovectores y Autovalores:**

La ecuación de autovectores y autovalores para esta matriz se expresa como:

$$S_{\text{intra}}^{-1}S_{\text{entre}}\mathbf{w} = \lambda\mathbf{w} \quad (6)$$

Donde:

- $\mathbf{w}$ es un autovector que representa una dirección en el espacio de características.
- λ es el autovalor correspondiente a $\mathbf{w}$.

Los autovectores $\mathbf{w}$ definen las nuevas dimensiones en las que proyectamos los datos para optimizar la discriminación entre clases, mientras que los autovalores λ representan la magnitud de la separación entre clases en la dirección dada por su autovector correspondiente.

Para obtener los autovectores y autovalores, se implementaron transformaciones de Householder y rotaciones de Givens para reducir la matriz original a forma Hessenberg. Luego se aplicó un algoritmo QR con descomposición en rotaciones de Givens, que permitió obtener los autovalores y autovectores deseados.

- **Selección de los Mejores Autovectores:** Se seleccionaron los autovectores asociados a los mayores autovalores, ya que estas direcciones son las que más contribuyen a la separación entre las clases. En este caso, dado que los datos tienen dos clases, se selecciona el primer autovector más significativo para proyectar los datos a una dimensión, lo que mejora la discriminación entre las clases en el espacio reducido.

2.4. Proyección y Clasificación

- **Proyección de los Datos:** Una vez calculadas las direcciones que maximizan la separabilidad entre clases, se proyectaron los datos tanto del conjunto de entrenamiento como del conjunto de prueba sobre esas direcciones. Dado que en este caso solo se utilizó el primer autovector, la proyección se llevó a cabo de la siguiente forma:

$$X_{\text{proy}}^{(i)} = X_i \cdot \mathbf{w} \quad (7)$$

Donde:

- $\mathbf{w}$ es el autovector correspondiente a la matriz $S_{\text{intra}}^{-1}S_{\text{entre}}$ que maximiza la separabilidad entre clases.
- $X_{\text{proy}}^{(i)}$ es el valor proyectado de la muestra i en la dirección definida por $\mathbf{w}$.

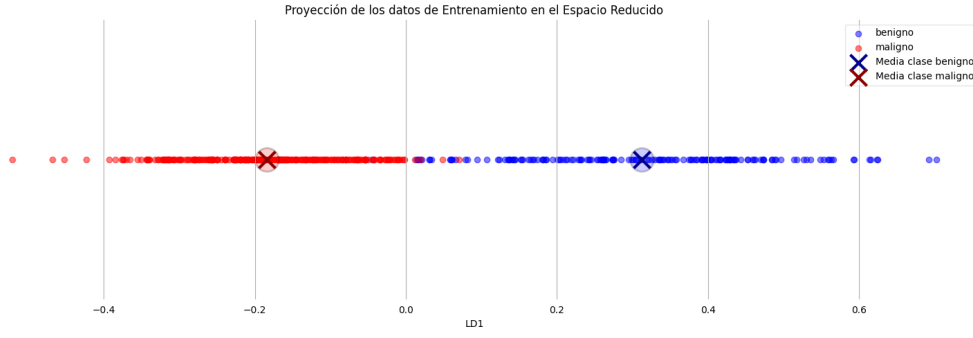


Figura 1: Proyección de los datos de entrenamiento en el espacio reducido por LDA.

- **Clasificación:** Para la clasificación de las muestras proyectadas, se utilizó un enfoque basado en la distancia a las medias proyectadas de cada clase. En este caso, las medias de las clases benigno (μ_0) y maligno (μ_1) en el espacio proyectado se calcularon como:

$$\mu_{\text{proy}}^{(0)} = \frac{1}{N_0} \sum_{i \in C_0} X_{\text{proy}}^{(i)} \quad \text{y} \quad \mu_{\text{proy}}^{(1)} = \frac{1}{N_1} \sum_{i \in C_1} X_{\text{proy}}^{(i)} \quad (8)$$

Donde:

- N_0 es el número de muestras de la clase benigno.
- N_1 es el número de muestras de la clase maligno.
- C_0 y C_1 son los conjuntos de índices correspondientes a las muestras de cada clase.

Para cada muestra del conjunto de prueba, se calcula la distancia a las medias de ambas clases proyectadas, y se asigna la clase correspondiente en función de la menor distancia. Es decir, si la distancia a la media de la clase benigno es menor que la distancia a la media de la clase maligno, la muestra se clasifica como benigno, y viceversa:

$$y_{\text{pred}}^{(i)} = \begin{cases} 0 & , \text{ si } |X_{\text{proy}}^{(i)} - \mu_{\text{proy}}^{(0)}| < |X_{\text{proy}}^{(i)} - \mu_{\text{proy}}^{(1)}| \\ 1 & , \text{ caso contrario} \end{cases} \quad (9)$$

- **Evaluación de Resultados:** Los resultados obtenidos se evaluaron utilizando el conjunto de prueba, comparando las clasificaciones predichas con las etiquetas reales. La precisión de la clasificación se calculó como:

$$\text{Exactitud} = \frac{\text{Número de clasificaciones correctas}}{\text{Total de muestras}} \times 100 \quad (10)$$

En la Figura 1 se muestra la proyección de los datos de entrenamiento en el espacio reducido generado por el Análisis Discriminante Lineal (LDA), junto con las medias de cada clases proyectada. En este gráfico, se puede observar cómo las dos clases (benigno y maligno) están agrupadas de manera clara, aunque hay un pequeño intervalo entre los grupos donde se ve un pequeño solapamiento de algunos pocos puntos. Aún así la distribución de los puntos en el espacio proyectado sugiere que la técnica de LDA ha sido adecuada para capturar las diferencias importantes entre las clases, lo que contribuye a la efectividad del modelo para la clasificación.

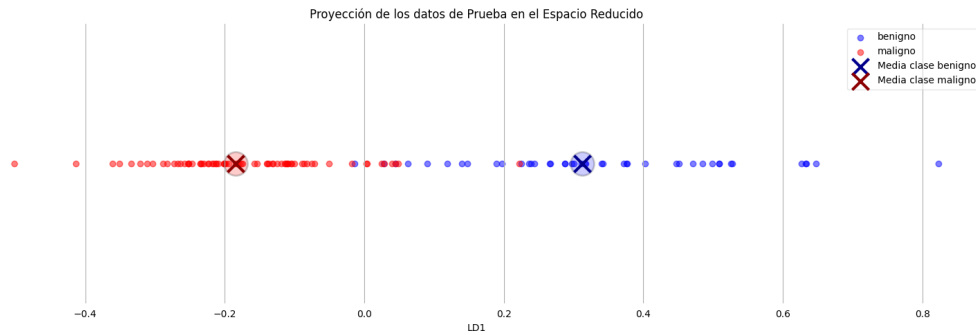


Figura 2: Proyección de los datos de prueba en el espacio reducido por LDA.

3. Conclusiones

La exactitud obtenida sobre el conjunto de prueba fue del 95.61 %. Esto significa que el clasificador basado en LDA es capaz de identificar correctamente una gran proporción de las muestras de prueba, demostrando el alto rendimiento del mismo.

A continuación, se muestra la proyección de los datos de prueba en el mismo espacio reducido, lo que permite visualizar cómo se distribuyen las muestras de prueba con respecto a las medias de las clases benigno y maligno. En este gráfico, se puede observar que la mayoría de las muestras se agrupan de acuerdo con su clase verdadera, lo que respalda la alta precisión del modelo.

En la Figura 2 se muestra la proyección de los datos de prueba. Vemos que los mismos se distribuyen de manera que la clasificación de cada punto es consistente con su clase. De este gráfico, podemos interpretar que el modelo ha sido capaz de generalizar correctamente a nuevos datos no vistos durante el entrenamiento.

En conclusión, el modelo ha mostrado un desempeño muy bueno tanto en los datos de entrenamiento como en los de prueba, con una alta precisión en la clasificación de los tumores como benignos o malignos, validando la utilidad del Análisis Discriminante Lineal para este tipo de tareas.

Referencias

- [1] S. Balakrishnama, A. Ganapathiraju. *Linear Discriminant Analysis - A Brief Tutorial*. Institute for Signal and Information Processing, Department of Electrical and Computer Engineering, Mississippi State University, Mississippi, 1998. [Link al artículo](#)

4. Apéndice

.1. Preprocesamiento de los Datos

A continuación se presenta el código utilizado para la división de los datos en conjuntos de entrenamiento y prueba, así como la estandarización de las características:

```
1 # Importación de librerías necesarias
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import train_test_split
4 import pandas as pd
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 # 1. Carga del dataset de cáncer de mama desde scikit-learn
10 data = load_breast_cancer()
11
12 # 2. Creación de un DataFrame con las características y nombres de columnas
13 df = pd.DataFrame(data.data, columns=data.feature_names)
14
15 # 3. Añadir la columna 'target' que indica si el tumor es benigno (0) o maligno (1)
16 df['target'] = data.target
17
18 # 4. Separar las características (X) del objetivo (y)
19 X = df.drop('target', axis=1).values # Matriz de características
20 y = df['target'].values # Vector objetivo
21
22 # 5. Dividir los datos en conjuntos de entrenamiento (80%) y prueba (20%)
23 X_train, X_test, y_train, y_test = train_test_split(
24     X, y, test_size=0.2, random_state=42
25 )
26
27 # 6. Calcular la media y la desviación estándar del conjunto de entrenamiento
28 X_train_mean = np.mean(X_train, axis=0)
29 X_train_std = np.std(X_train, axis=0)
30
31 # 7. Estandarizar los datos de entrenamiento y prueba utilizando la media y
32 # desviación estándar calculadas en el conjunto de entrenamiento
33 X_train_standarizado = (X_train - X_train_mean) / X_train_std
34 X_test_standarizado = (X_test - X_train_mean) / X_train_std
```

Código 1: División de datos y estandarización

.2. Cálculo de las Matrices de Dispersión

En esta sección se muestra el código utilizado para calcular las matrices de dispersión dentro de las clases y entre las clases.

```
1 # 1. Calcular la media global de los datos estandarizados del conjunto de entrenamiento
2 media_global = np.mean(X_train_standarizado, axis=0)
3
4 # 2. Separar los datos del conjunto de entrenamiento por clases
5 # clase_0: Tumores benignos (y = 0)
```

```

6  #     clase_1: Tumores malignos (y = 1)
7  clase_0 = X_train_standarizado[y_train == 0]
8  clase_1 = X_train_standarizado[y_train == 1]
9
10 # 3. Calcular la media de cada clase
11 media_0 = np.mean(clase_0, axis=0)
12 media_1 = np.mean(clase_1, axis=0)
13
14 # 4. Inicializar la matriz de dispersin intra-clase (S_intra) con ceros
15 S_intra = np.zeros((X_train.shape[1], X_train.shape[1]))
16
17 # 5. Calcular la matriz de dispersin intra-clase (S_intra)
18 S_intra += (clase_0 - media_0).T @ (clase_0 - media_0)
19 S_intra += (clase_1 - media_1).T @ (clase_1 - media_1)
20
21 # 6. Calcular el tamao de cada clase (nmero de muestras)
22 n_0 = clase_0.shape[0]
23 n_1 = clase_1.shape[0]
24
25 # 7. Inicializar la matriz de dispersin entre-clases (S_entre) con ceros
26 S_entre = np.zeros((X_train.shape[1], X_train.shape[1]))
27
28 # 8. Calcular la matriz de dispersin entre-clases (S_entre)
29 S_entre += n_0 * np.outer((media_0 - media_global), (media_0 - media_global))
30 S_entre += n_1 * np.outer((media_1 - media_global), (media_1 - media_global))

```

Código 2: Cálculo de matrices de dispersión

.3. Búsqueda de las Direcciones que Maximicen la Separabilidad

El siguiente código muestra cómo se resuelve el problema de optimización para encontrar las direcciones que maximizan la separabilidad entre las clases utilizando los autovectores de la matriz $S_{\text{intra}}^{-1} S_{\text{entre}}$.

```

1
2 def sol_trinffil(A, b):
3     n = len(b)
4     x = np.zeros(n)
5     for i in range(n):
6         x[i] = (b[i] - A[i][:i] @ x[:i]) / A[i][i]
7     return x
8
9 def sol_trsupfil(A, b):
10    n = len(b)
11    x = np.zeros(n)
12    for i in reversed(range(n)):
13        x[i] = (b[i] - A[i][i+1:] @ x[i+1:]) / A[i][i]
14    return x
15
16 def dlup(A):
17     A = np.array(A, dtype=float)
18     n = A.shape[0]
19     P = np.eye(n)

```



```

20     for k in range(n-1):
21         l = np.argmax(np.abs(A[k:, k])) + k
22         if A[l, k] != 0 and l != k:
23             A[[k, l]], P[[k, l]] = A[[l, k]], P[[l, k]]
24             A[k+1:, k] /= A[k, k]
25             A[k+1:, k+1:] -= np.outer(A[k+1:, k], A[k, k+1:])
26     L = np.tril(A, k=-1) + np.eye(n)
27     U = np.triu(A)
28     return L, U, P
29
30 def inv_lu(A):
31     n = A.shape[0]
32     L, U, P = dlup(A)
33     I = np.eye(n)
34     A_inv = np.zeros((n, n))
35     for i in range(n):
36         y = sol_trinffil(L, P @ I[:, i])
37         A_inv[:, i] = sol_trsupfil(U, y)
38     return A_inv
39
40 def house(x):
41
42     n = len(x)
43     rho = 0
44     u = x.copy()
45     u[0] = 1.
46     mu = 0
47
48     if n == 1:
49         sigma = 0
50     else:
51         sigma = np.sum(x[1:]**2)
52
53     if sigma>0 or x[0]<0:
54         mu = np.sqrt(x[0]**2 + sigma)
55
56     if x[0]<=0:
57         gamma = x[0] - mu
58     else:
59         gamma = -sigma/(x[0] + mu)
60
61     rho = 2*gamma**2/(gamma**2 + sigma)
62     u = u/gamma
63     u[0] = 1
64
65     return u, rho
66
67 def givens(x1,x2):
68
69     c = 1.
70     s = 0.
71     ax1 = abs(x1)
72     ax2 = abs(x2)

```

```

73
74     if ax1 + ax2 > 0:
75         if ax2 > ax1:
76             tau = -x1/x2
77             s = -np.sign(x2)/np.sqrt(1 + tau**2)
78             c = tau*s
79         else:
80             tau = -x2/x1
81             c = np.sign(x1)/np.sqrt(1 + tau**2)
82             s = tau*c
83
84     return c, s
85
86
87 def fhess(A, p=1):
88     m, n = A.shape
89     if m != n:
90         print("La matriz no es cuadrada")
91         return None
92     Q = np.eye(m)
93     H = A.copy()
94
95     if p == 0:
96         for j in range(n-2):
97             u, rho = house(H[j+1:, j])
98             w = rho * u
99             H[j+1:, j:] = H[j+1:, j:] - np.outer(w, u.T @ H[j+1:, j:])
100            H[:, j+1:] = H[:, j+1:] - H[:, j+1:]@ np.outer(w, u.T)
101            Q[:, j+1:] = Q[:, j+1:] - Q[:, j+1:]@np.outer(w, u.T)
102
103     elif p == 1:
104         for j in range(n - 2):
105             for i in range(j + 2, n):
106                 c, s = givens(H[j + 1, j], H[i, j])
107                 rot = np.array([[c, -s], [s, c]])
108                 H[[j + 1, i], j:] = rot @ H[[j + 1, i], j:]
109                 H[:, [j + 1, i]] = H[:, [j + 1, i]] @ rot.T
110                 Q[:, [j + 1, i]] = Q[:, [j + 1, i]] @ rot.T
111     else:
112         print("Elegir un p que sea 0 o 1")
113         return None
114
115     return Q, H
116
117 def autqr(A, err= 1e-10, M = 100):
118     n = A.shape[0]
119     Q, H = fhess(A)
120
121     G_rot = np.zeros((n, 2))
122
123     for k in range(M):
124         for j in range(n-1):
125             c, s = givens(H[j, j], H[j+1, j])

```

```

126     G_rot[j, :] = np.array([c, s])
127     G_cs = np.array([[c, -s], [s, c]])
128     H[[j, j+1], j:] = G_cs @ H[[j, j+1], j:]
129
130     for l in range(n-1):
131         c, s = G_rot[l, :]
132         G_cs = np.array([[c, -s], [s, c]])
133         H[:, [l, l+1]] = H[:, [l, l+1]] @ G_cs.T
134         Q[:, [l, l+1]] = Q[:, [l, l+1]] @ G_cs.T
135
136     if np.linalg.norm(H - np.diag(np.diag(H)), 'fro') < err:
137         print ("llegamos a la tolerancia")
138         break
139
140     return Q, H
141
142 # Calcular la matriz inversa de S_intra y hallar autovalores y autovectores
143 S_intra_inv = inv_lu(S_intra)
144 Q, H = autqr(S_intra_inv @ S_entre)
145 eigvals = np.diag(H)
146 eigvecs = Q
147
148 # Ordenar autovectores segn autovalores en orden descendente
149 indices_ordenados = np.argsort(eigvals)[::-1]
150 eigvecs = eigvecs[:, indices_ordenados]
151
152 # Seleccionar el autovector con el mayor autovalor para la proyecin (LDA)
153 W = eigvecs[:, :1]

```

Código 3: Cálculo de autovectores y valores propios

4. Proyección y Clasificación

El siguiente código muestra cómo se proyectan los datos en el espacio reducido generado por LDA y se realiza la clasificación en función de las distancias a las medias proyectadas de cada clase.

```

1 # Proyección de los datos de entrenamiento y prueba
2 X_train_proyec = X_train_standarizado @ W
3 X_test_proyec = X_test_standarizado @ W
4
5 # Medias proyectadas por clase
6 media_proyec_0 = np.mean(X_train_proyec[y_train == 0])
7 media_proyec_1 = np.mean(X_train_proyec[y_train == 1])
8
9 # Clasificador usando distancia a las medias proyectadas
10 def clasificador(x, mean_0, mean_1):
11
12     clase = 1
13
14     if np.abs(x - mean_0) < np.abs(x - mean_1):
15         clase = 0
16

```

```
17     return clase
18
19 # Predicciones para el conjunto de prueba
20 y_pred = np.array([clasificador(x) for x in X_test_proyec])
21
22 # Clculo de la exactitud
23 exactitud = np.mean(y_pred == y_test)
24 print(f'Exactitud: {exactitud * 100:.2f}%')
```

Código 4: Proyección y clasificación